



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

Advanced Database Systems (INFS3200)

Lecture 3: Distributed Query Processing

Lecturer: Dr Junliang Yu

School of Electrical Engineering and Computer Science (EECS)

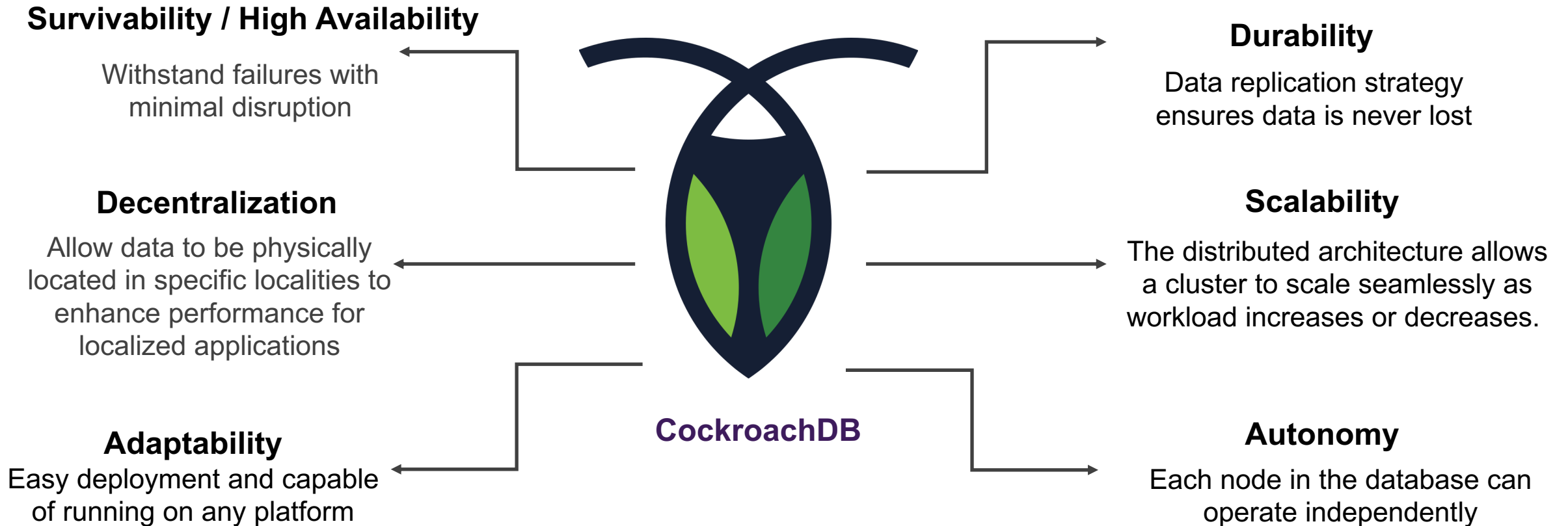
Faculty of Engineering, Architecture and Information Technology

The University of Queensland

Last Week: DDB Design

- Top-down and bottom-up design approaches
- DDB has a Global Conceptual Schema
- Centralized vs distributed – understanding of main differences
- Transparency levels and problems associated with them
- Data fragmentation, replication and allocation
- Properties of Fragmentation: *Completeness, Disjointness, Reconstruction* (**CDR**)
- *Minterm* predicates are used to maintain CDR
- *Semi-Join* operator can be used in fragmentation
- *SSOT* is a property of DDB.

Review: Essential features of Distributed Database systems

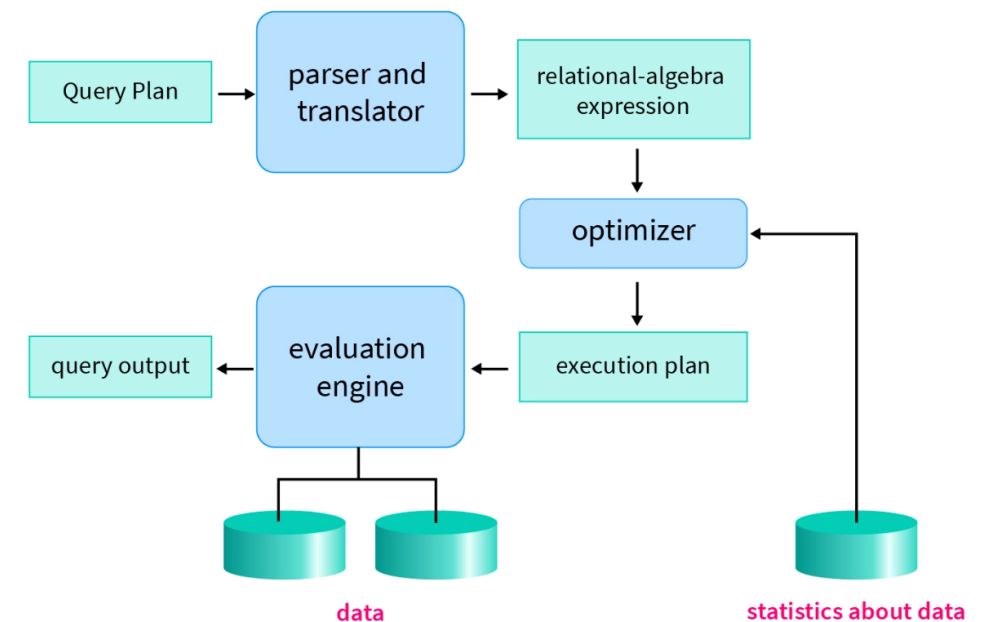


Outline

- What is and Why Distributed Query Processing (DQP)
- DQP framework and Terms
 - Query Decomposition
 - Data Localization
 - Query Optimization Process
 - Search Strategies
 - Search Space and Solution Space
 - Cost in Distributed Query Processing
- Reduction in Data Localization
 - Horizontal (Selection & Join) Fragment
 - Vertical Fragment
- Join ordering and Semi-Join in DQP

What is Query Processing?

- **Query processing** refers to the sequence of steps performed by a DBMS to execute a query and retrieve the desired data efficiently. It involves translating a high-level query (e.g., in SQL) into a set of low-level operations that the database can execute.
- **Parsing and Translation:** The query is checked for syntax errors and then converted into an internal format, such as relational algebra expression.
- **Optimization:** The DBMS explores multiple strategies for executing the query and selects the most efficient one. This step often involves cost estimation (e.g., I/O, CPU, etc.). The optimal strategy is converted into a sequence of physical operations (e.g., joins).
- **Execution:** The DBMS runs the chosen plan and returns the result.



Query Processing Example

- **Query** `SELECT name, age FROM students
WHERE grade > 85 ORDER BY age DESC;`
- **Parsing and Translation** $\rightarrow \pi_{\text{name, age}}(\sigma_{\text{grade} > 85}(\text{students}))$
- **Optimization:** $\rightarrow$ Index scanning if an index on grade exists.
- **Explore execution strategies and generate the optimal query execution plan:**
 - **Step 1:** Perform an **index scan** on students for grade > 85.
 - **Step 2:** **Project** only the name and age attributes.
 - **Step 3:** Use a **heap sort** to order by age DESC.
- **Execution:** The DBMS runs the chosen plan and returns the result.

Why Distributed Query Processing?

- **Query Processing (QP)** determines the performance in both centralised and distributed DBMSs.
- Unfortunately, QP in distributed environments is **much more difficult**:
 - the relations (tables) involved in a distributed query may be **fragmented** and/or **replicated**, thereby inducing costs (e.g. communication and computation costs).
 - query response time may become very high across **many sites**.

Example:

```
EMP (ENO, ENAME, TITLE)
ASG (ENO, PNO, RESP, DUR)
```

“Find the names of employees who are managing a project”



```
SELECT ENAME
FROM EMP NATURAL JOIN ASG
WHERE RESP = "Manager"
```

Strategy A: $\Pi_{ENAME}(\sigma_{RESP="Manager"} \wedge EMP.ENO=ASG.ENO (EMP \times ASG))$

Strategy B: $\Pi_{ENAME}(\bowtie_{ENO} (\sigma_{RESP="Manager"} (ASG)))$

Which Better?

Why Distributed Query Processing?

Example:

EMP (ENO, ENAME, TITLE)
ASG (ENO, PNO, RESP, DUR)



$EMP_1 = \sigma_{ENO \leq "E3"}(EMP)$

@ Site 3

$EMP_2 = \sigma_{ENO > "E3"}(EMP)$

@ Site 4

$ASG_1 = \sigma_{ENO \leq "E3"}(ASG)$

@ Site 1

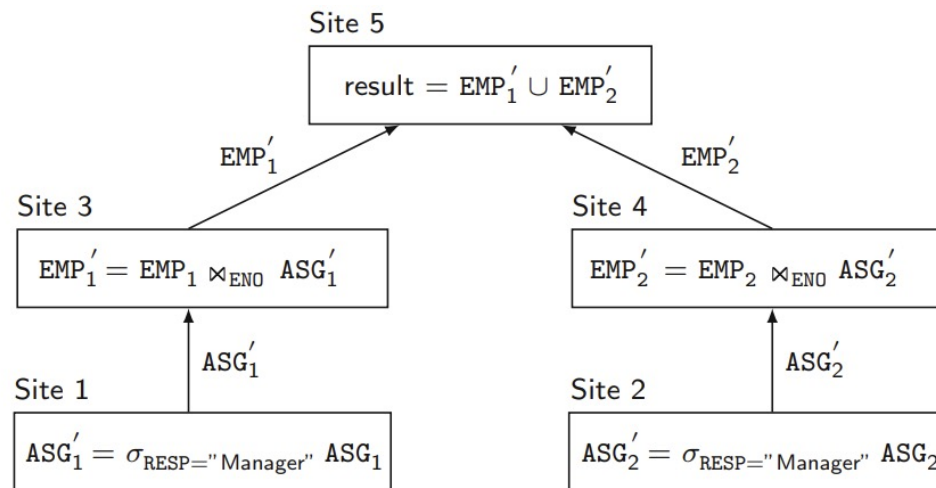
$ASG_2 = \sigma_{ENO > "E3"}(ASG)$

@ Site 2

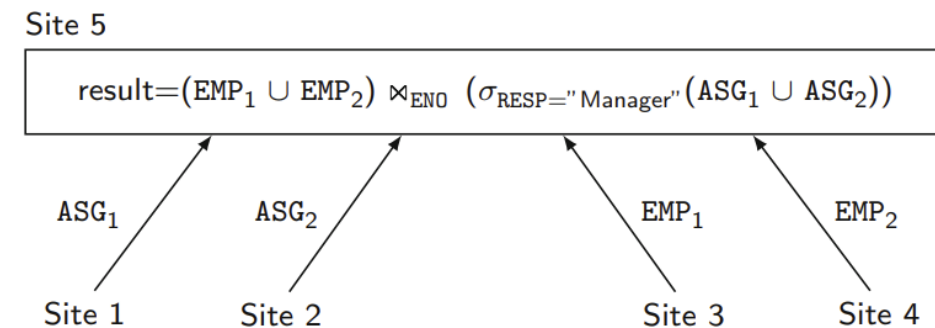
Result

@ Site 5

Strategy B: $\Pi_{ENAME}(\bowtie_{ENO}(\sigma_{RESP="Manager"}(ASG)))$



(a)

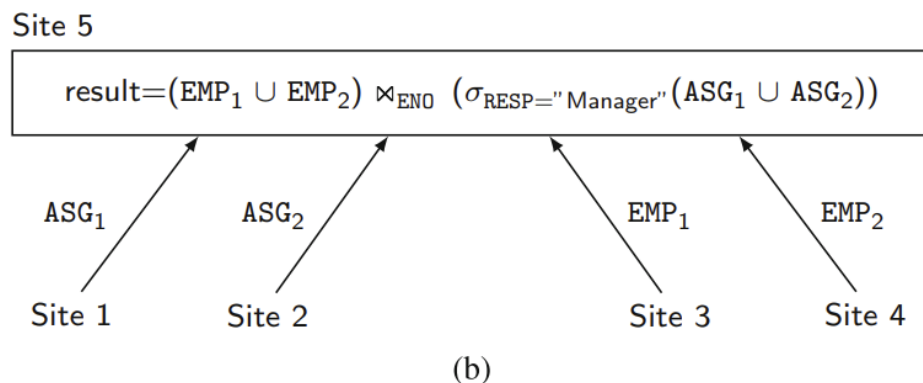
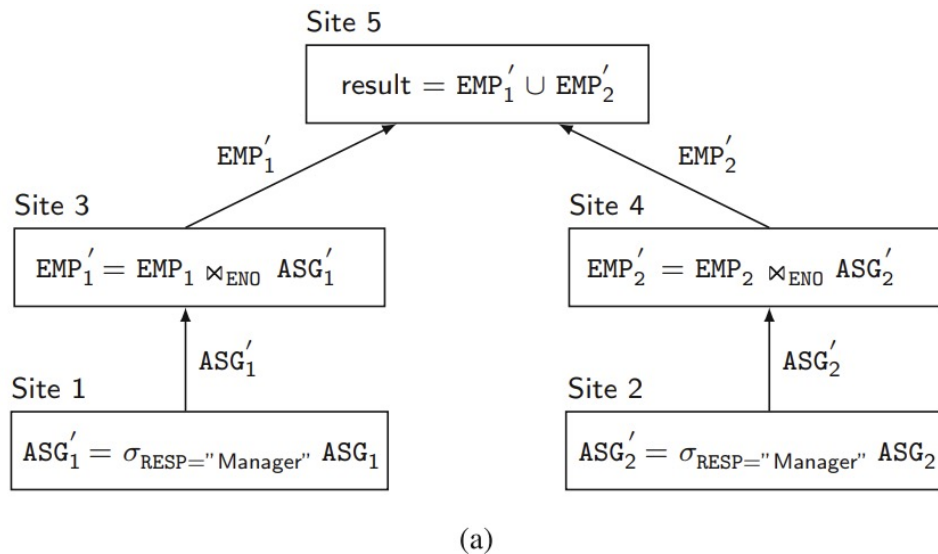


(b)

Which Better?

Why Distributed Query Processing?

Prior Knowledge on DDBs
 $tupacc = 1$, $tuptrans = 10$
 400 tuples in EMP and 1000 in ASG
 There are 20 managers and data is uniformly distributed among sites.



The total cost of strategy A can be derived as follows:

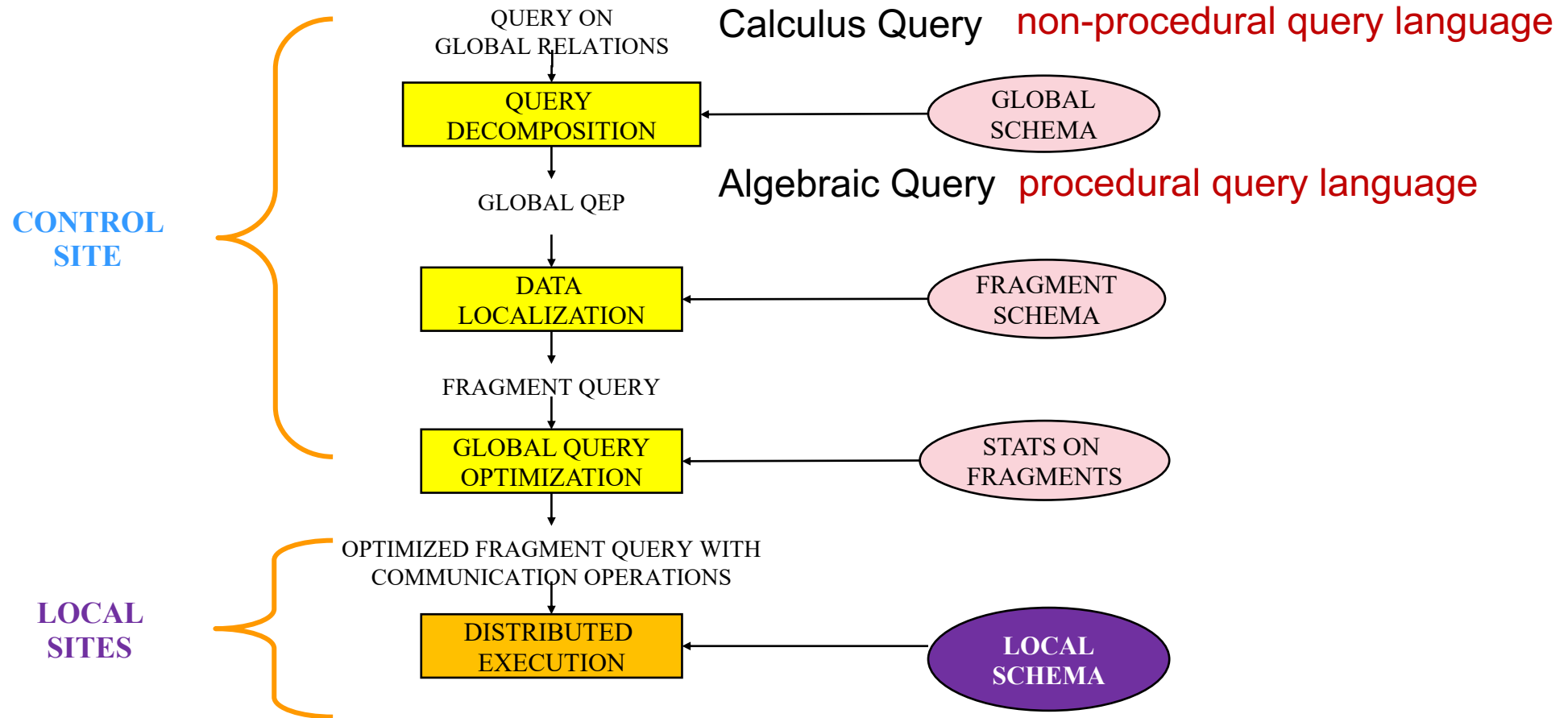
1. Produce ASG' by selecting ASG requires $(10 + 10) * tupacc = 20$
 2. Transfer ASG' to the sites of EMP requires $(10 + 10) * tuptrans = 200$
 3. Produce EMP' by joining ASG' and EMP requires $(10 + 10) * tupacc * 2 = 40$
 4. Transfer EMP' to result site requires $(10 + 10) * tuptrans = 200$
- The total cost is $\overline{460}$

The cost of strategy B can be derived as follows:

1. Transfer EMP to site 5 requires $400 * tuptrans = 4,000$
 2. Transfer ASG to site 5 requires $1000 * tuptrans = 10,000$
 3. Produce ASG' by selecting ASG requires $1000 * tupacc = 1,000$
 4. Join EMP and ASG' requires $400 * 20 * tupacc = 8,000$
- The total cost is $\overline{23,000}$

Which Better?

Layers of Query Processing



...What is a Control Site?

General Query Decomposition

Output example:

$\Pi_{ENAME}(EMP \bowtie_{ENO} (\sigma_{RESP="Manager"}(ASG)))$

There are **4 steps** to decompose a query:

- 1. Normalization:** convert the query to a standard form
 - SQL is based on **relational calculus**, which is non-procedural, in the form of $\{t \mid F(t)\}$. **Example:** $\{ T \mid T \in \text{Students} \wedge T.\text{grade} > 85 \}$
 - Relational algebra**, a step-by-step program using set operations (σ , π , $\bowtie$, $\cap$, $\cup$, $\times$,...)
- 2. Analysis:** make sure it is semantically correct (i.e. contribute to the result)
- 3. Simplification:** remove redundant predicates
- 4. Rewriting:** generate a “good” GLOBAL algebraic query (i.e., ready for optimization)

Relational Calculus vs Relational Algebra?

Data Localization for Global QEP

Input: Algebraic query on distributed relations (global QEP)

Output: localized queries (Local QEP)

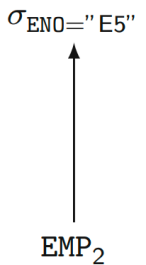
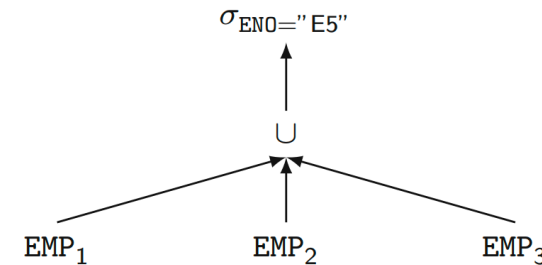
Two steps

- **Use fragments** to replace relations in the query
- **Simplify** and restructure to get another “good” query

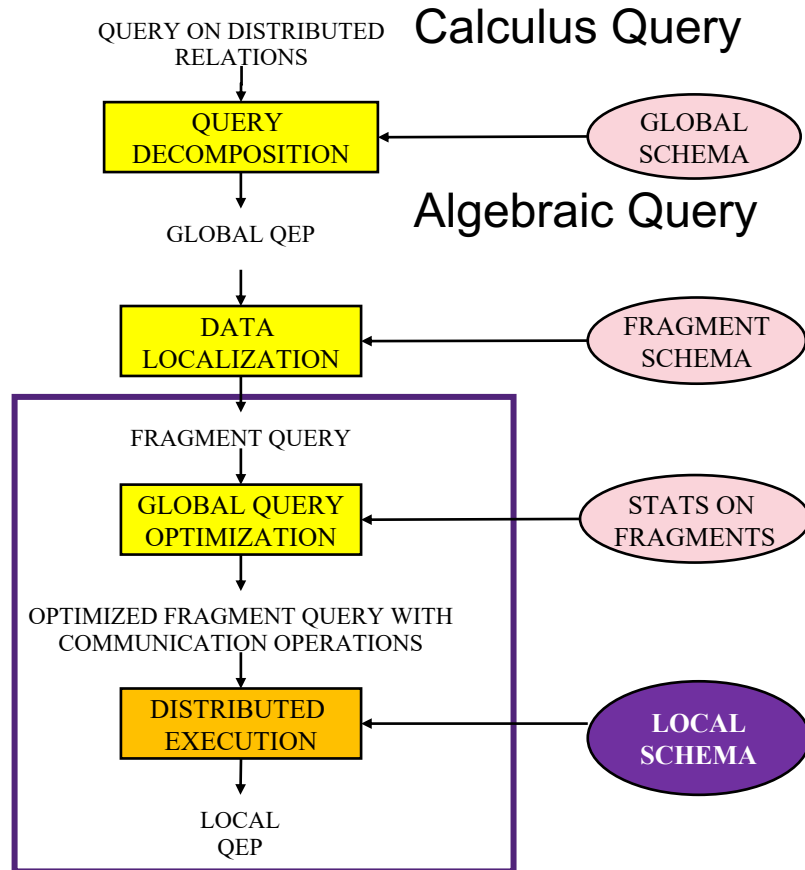
Notes

- May need to add **union** ($\cup$) and **join** ($\bowtie$) operations
 - Reorder operations: *push up* $\cup$ and $\bowtie$, *push down* σ (examples later)
 - Simplify operations: *eliminate unnecessary operations*

$$\Pi_{ENAME}(\text{EMP} \bowtie_{ENO} (\sigma_{RESP="Manager"}(\text{ASG})))$$

$$\text{EMP} = \text{EMP}_1 \cup \text{EMP}_2 \cup \text{EMP}_3$$


Query Optimization Process



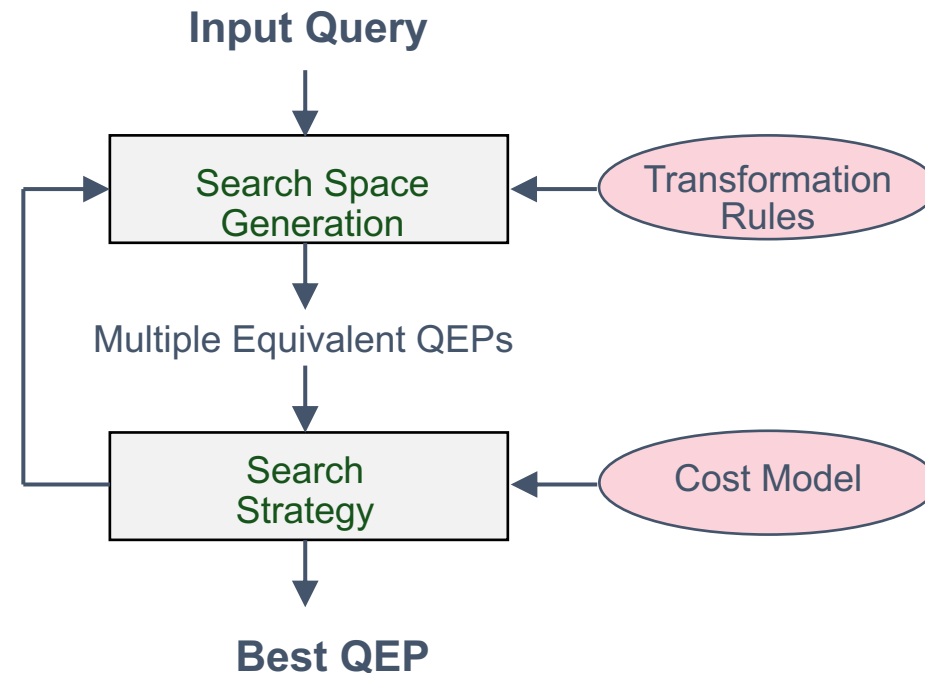
Associativity of the Join

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

Commutativity of the Join

$$R \bowtie S = S \bowtie R$$

General Query Optimization Process

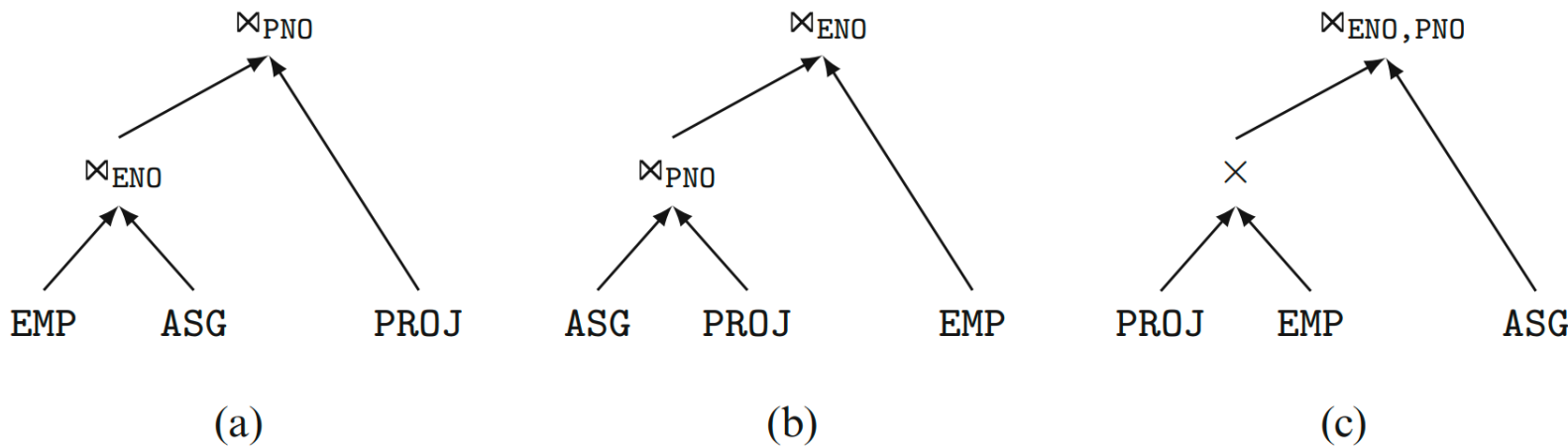


Query Optimization Process

For a given query, the **search space** can be defined as the set of equivalent operator trees that can be produced using transformation rules.

```
SELECT ENAME, RESP
FROM EMP NATURAL JOIN ASG NATURAL JOIN PROJ
```

The number of alternative join trees that can be produced by applying the commutativity and associativity rules is **$O(N!)$** for N relations.



Equivalent join trees

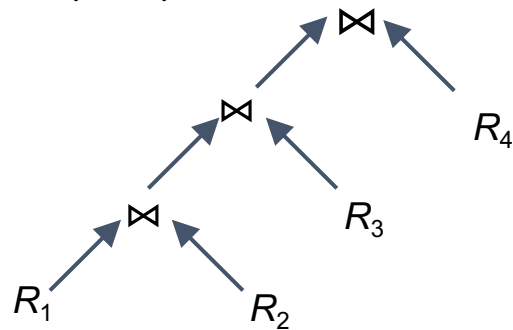
$$\text{PROJ} \bowtie_{\text{PNO}} \text{ASG} \bowtie_{\text{ENO}} \text{EMP}$$

Shape of Join Tree

To narrow down the search space by restricting the shape of Join trees

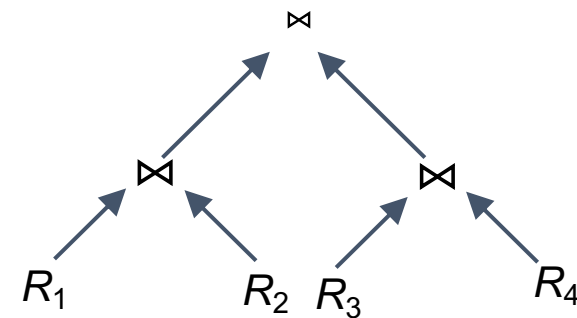
- **Linear** (left-deep) join trees vs. **Bushy** join trees
- A linear tree is a tree such that at least one operand of each operator node is a base relation (fragment).
- A bushy tree is more general and may have operators with no base relations as operands.

(Left) **Linear Join Tree**



The size of the search space is reduced to $O(2^N)$.

Bushy Join Tree



Example: how do we join 4 (R_1, R_2, R_3, R_4) fragments together?

Two Types of Search Strategies

Exhaustive search and evaluation on all participating data and system components (**Model based**)

- Cost model based
- Optimal (+)
- Hard to have all system performance data required by cost models (-)
- Combinatorial complexity in the number of relations (-)

Heuristics (**Rule based**)

- Not optimal
- Perform **unary** operations e.g., **selection**, **projection** **first**, ...
- Avoid Cartesian products
- Reorder operations to **reduce intermediate** relation size
- Replace a join by a series of **semi-joins**?

Leveraging these rules to reduce the size of intermediate results and the transmission cost.

Solution Space

A refined subspace of the search space that contains only viable algebraic expressions (subtrees) to further restrict the Search Space

Cost functions (in terms of **time** or **size** of data)

- Total processing cost or total end-to-end response time
- These might have different weights in different distributed environments (**LAN vs WAN**).
- Can also **maximize** system **throughput**

Search algorithm

- How do we move inside the solution space?
- Exhaustive search, **heuristic algorithms** (iterative improvement, simulated annealing, genetic,...)

Options of Cost Functions

Total Cost

- The **sum of all times** incurred in processing the operators of the query at various sites and in intersite communication
- Reduce each cost (in terms of time) component **individually**
- Do as little of each cost component as possible

What would be the best we could do?

Response Time

- The **time elapsed** for executing the query
- Do as many things as possible in **parallel**
- *May increase total time because of increased total activity*

response time of a query may be **significantly less** than its **total cost**

Total Cost

Summary of all cost factors

Total Cost = CPU cost + I/O cost + Communication cost

- **CPU Cost** = unit instruction cost * #instructions
- **I/O Cost** = unit disk I/O cost * #disk I/Os
- **Communication Cost** = message initiation + transmission

So, what can we do w.r.t. above three options?

Response Time

Elapsed time between the initiation and the completion (end-to-end) of a query (aka **latency**)

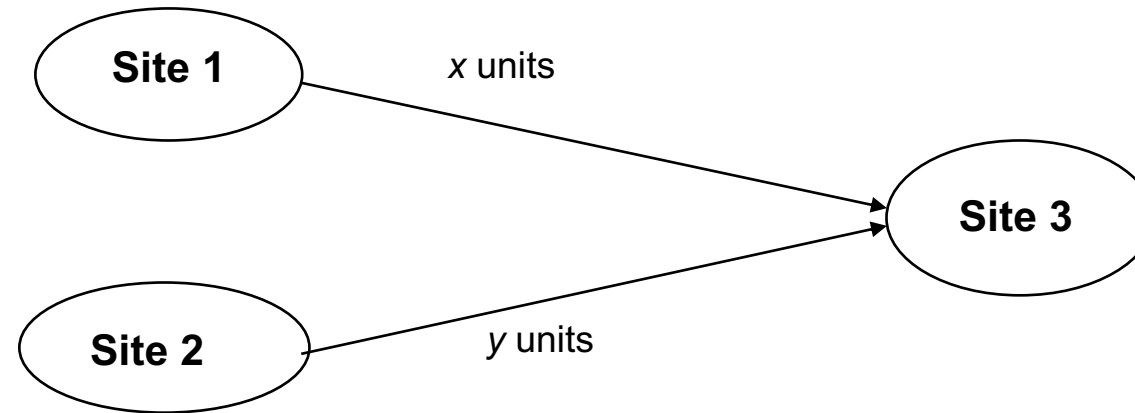
$$\text{Response time} = \text{CPU time} + \text{I/O time} + \text{Communication time}$$

- **CPU time** = **unit** instruction time * # **sequential** instructions
- **I/O time** = **unit** I/O time * # **sequential** I/Os
- **Communication time** = **unit** msg initiation time * # **sequential** msg
+ **unit** transmission time * # **sequential** bytes

So, what can we do w.r.t. above three options?

More details on Page 250, Chapter 8, Ozsú & Valduriez

Example: Total cost vs Response time

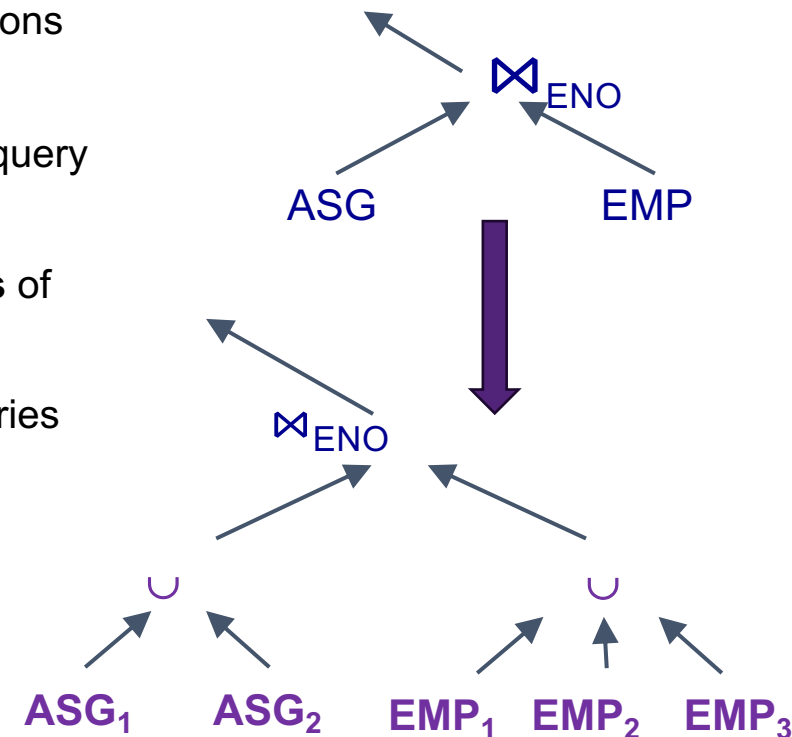
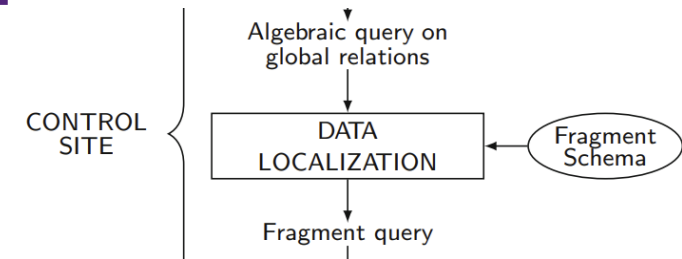


Assume only communication cost is considered:

- **Total cost** = 2 * message initialization time + unit transmission time * **(x+y)**
- **Response time** = **max** {time to send *x* from 1 to 3, time to send *y* from 2 to 3}
 - *time to send x from 1 to 3 = message initialization time + unit transmission time * x*
 - *time to send y from 2 to 3 = message initialization time + unit transmission time * y*

Reduction Techniques in Data Localization

- Data localization translates **an algebraic query** on global relations into a **fragment query** using information stored in the fragment schema.
- A naive way to do this is to generate a query where each global relation is substituted by its **materialization program**.
- **Materialization program** in distributed databases is a set of relational algebra operations that reconstruct a global relation from its fragments
 - This can be viewed as replacing the leaves of the operator tree of the distributed query with subtrees corresponding to the materialized programs.
 - this approach is inefficient because important **restructurings** and **simplifications** of the fragment query can still be made.
- **Reduction Techniques** can be used to generate simpler and optimized fragment queries
 - Reduction for (Primary) Horizontal Fragments
 - Reduction for Vertical Fragments
 - Reduction for Derived Fragments and Hybrid Fragments



Primary Horizontal Fragmentation - Revisited

- The **horizontal fragmentation** function distributes a relation based on **selection predicates**.

EMP(ENO, ENAME, TITLE)
ASG(ENO, PNO, RESP, DUR)

$$EMP_1 = \sigma_{ENO \leq "E3"}(EMP)$$

$$EMP_2 = \sigma_{"E3" < ENO \leq "E6"}(EMP)$$

$$EMP_3 = \sigma_{ENO > "E6"}(EMP)$$

$$ASG_1 = \sigma_{ENO \leq "E3"}(ASG)$$

$$ASG_2 = \sigma_{ENO > "E3"}(ASG)$$

ENO	ENAME	TITLE
E1	J. Doe	Elect. Eng
E2	M. Smith	Syst. Anal.
E3	A. Lee	Mech. Eng.
E4	J. Miller	Programmer
E5	B. Casey	Syst. Anal.
E6	L. Chu	Elect. Eng.
E7	R. Davis	Mech. Eng.
E8	J. Jones	Syst. Anal.

ENO	PNO	RESP	DUR
E1	P1	Manager	12
E2	P1	Analyst	24
E2	P2	Analyst	6
E3	P3	Consultant	10
E3	P4	Engineer	48
E4	P2	Programmer	18
E5	P2	Manager	24
E6	P4	Manager	48
E7	P3	Engineer	36
E8	P3	Manager	40

Reduction for HF: Selection in Union

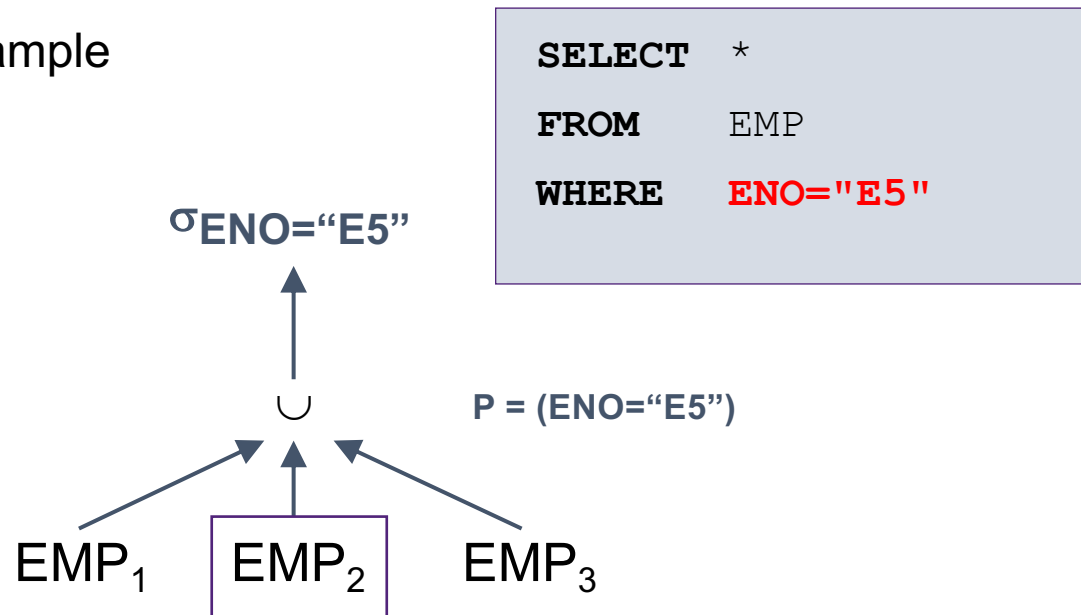
Reduction with **selection** operator (**deleting unnecessary** $\cup$)

- For a Relation R , there exists fragmentation:

$$F_R = \{R_1, R_2, \dots, R_w\}, \text{ where } R_j = \sigma_{p_j}(R) \text{ and}$$

Rule 1: $\sigma_{p_i}(R_j) = \emptyset$, if $\forall x \text{ in } R: \neg(p_i(x) \wedge p_j(x))$.

- Example



For two predicates, there does not exist an x in R that satisfies both predicates.



p_i and p_j are contradicting

- $EMP_1 = \sigma_{ENO \leq "E3"}(EMP)$
- $EMP_2 = \sigma_{"E3" < ENO \leq "E6"}(EMP)$
- $EMP_3 = \sigma_{ENO > "E6"}(EMP)$

$\sigma_{ENO="E5"}$

EMP_2

Reduced Query

Reduction for HF: Join

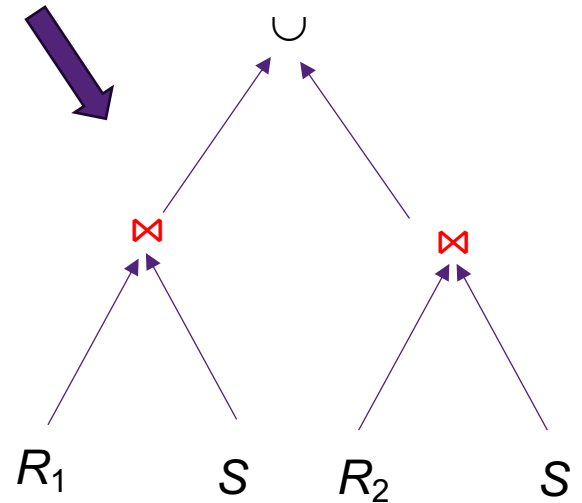
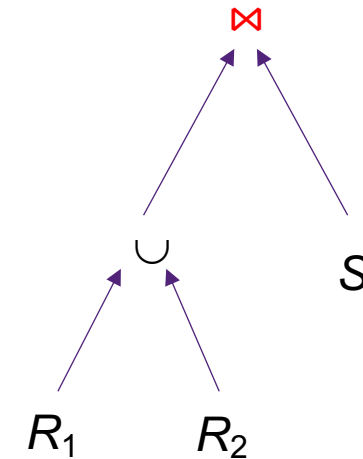
Reduction with **join** operator (deleting unnecessary $\bowtie$)

- Possible when fragmentation is done on join attributes
- Distribute join over union

$$(R_1 \cup R_2) \bowtie S \Leftrightarrow (R_1 \bowtie S) \cup (R_2 \bowtie S)$$

- Given $R_i = \sigma_{p_i}(R)$ and $R_j = \sigma_{p_j}(R)$

Rule 2: $R_i \bowtie R_j = \emptyset$, if $\forall x \text{ in } R_i, \forall y \text{ in } R_j: \neg(p_i(x) \wedge p_j(y))$



**unions can be moved
up in the operator tree**

Example: Reduction with join for HF

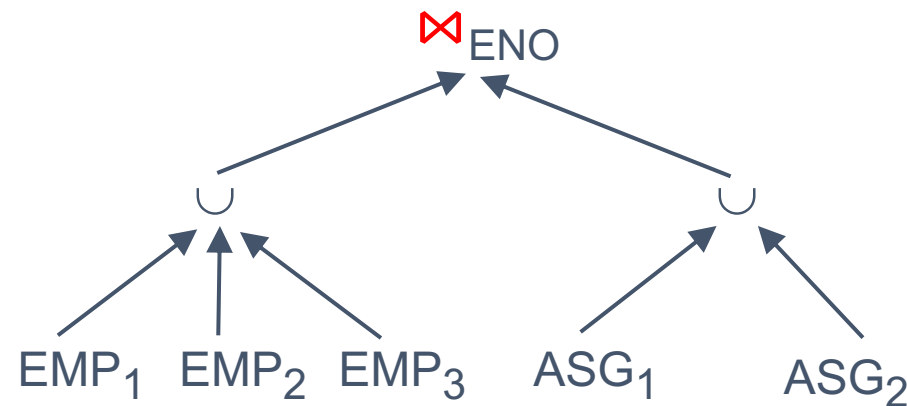
Assume EMP is fragmented as before and

- $ASG_1: \sigma_{ENO \leq "E3"}(ASG)$
- $ASG_2: \sigma_{ENO > "E3"}(ASG)$

- $EMP_1 = \sigma_{ENO \leq "E3"}(EMP)$
- $EMP_2 = \sigma_{"E3" < ENO \leq "E6"}(EMP)$
- $EMP_3 = \sigma_{ENO > "E6"}(EMP)$

Consider the query

```
SELECT      *
FROM    EMP, ASG
WHERE    EMP.ENO=ASG.ENO
```



Example: Distribute Join over Unions

Distribution of join over union: $(R_1 \cup R_2) \bowtie S \Leftrightarrow (R_1 \bowtie S) \cup (R_2 \bowtie S)$

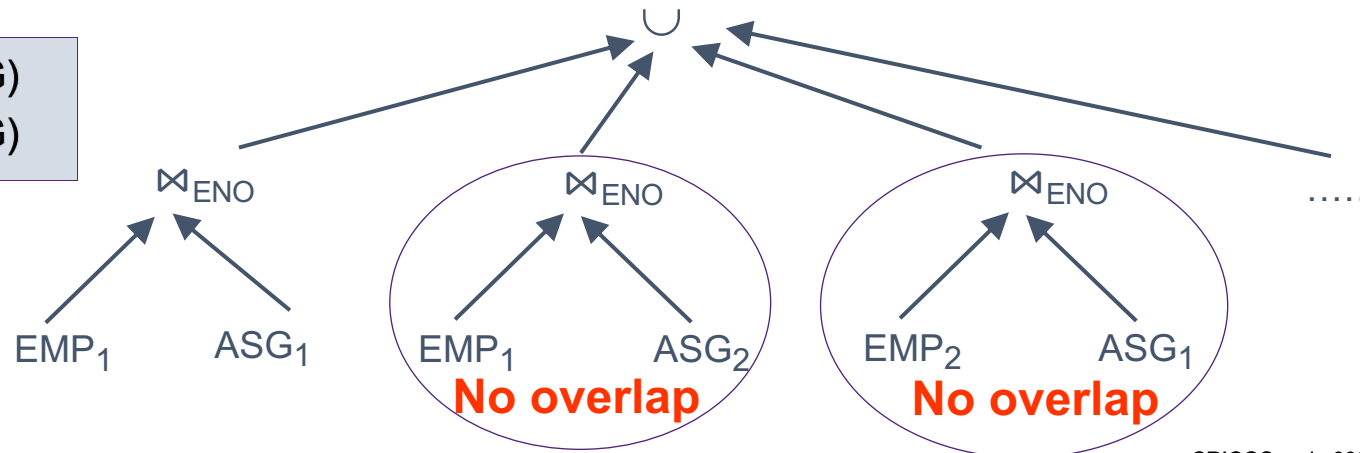
$$(R_1 \cup R_2) \bowtie (S_1 \cup S_2) \Leftrightarrow (R_1 \bowtie S_1) \cup (R_1 \bowtie S_2) \cup (R_2 \bowtie S_1) \cup (R_2 \bowtie S_2)$$

$$(\mathbf{EMP}_1 \cup \mathbf{EMP}_2 \cup \mathbf{EMP}_3) \bowtie (\mathbf{ASG}_1 \cup \mathbf{ASG}_2) =$$

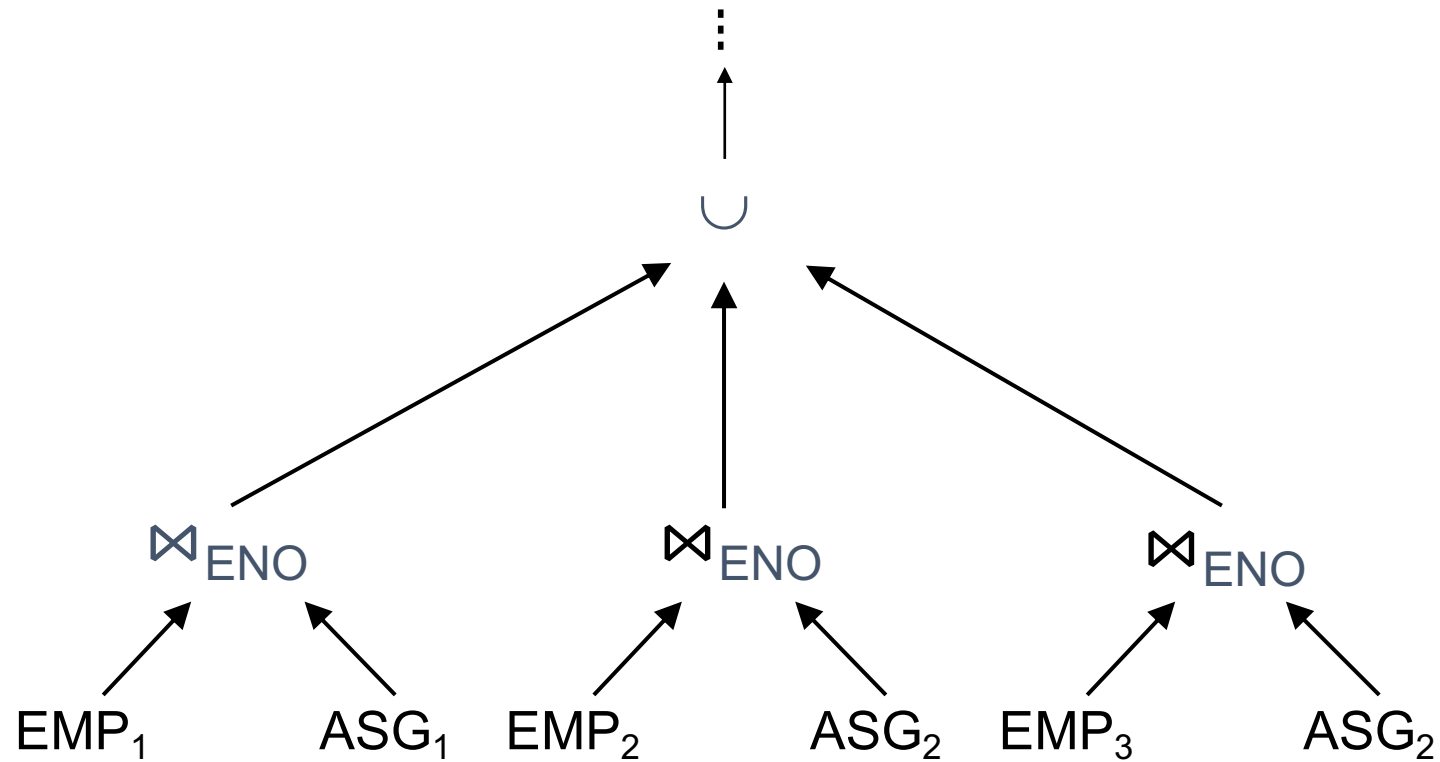
$$\begin{aligned} & \mathbf{EMP}_1 \bowtie \mathbf{ASG}_1 \cup \mathbf{EMP}_1 \bowtie \mathbf{ASG}_2 \cup \\ & \mathbf{EMP}_2 \bowtie \mathbf{ASG}_1 \cup \mathbf{EMP}_2 \bowtie \mathbf{ASG}_2 \cup \\ & \mathbf{EMP}_3 \bowtie \mathbf{ASG}_1 \cup \mathbf{EMP}_3 \bowtie \mathbf{ASG}_2 \end{aligned}$$

- $\mathbf{ASG}_1$: $\sigma_{\text{ENO} \leq \text{"E3"}}(\mathbf{ASG})$
- $\mathbf{ASG}_2$: $\sigma_{\text{ENO} > \text{"E3"}}(\mathbf{ASG})$

- $\mathbf{EMP}_1 = \sigma_{\text{ENO} \leq \text{"E3"}}(\mathbf{EMP})$
- $\mathbf{EMP}_2 = \sigma_{\text{"E3"} < \text{ENO} \leq \text{"E6"}}(\mathbf{EMP})$
- $\mathbf{EMP}_3 = \sigma_{\text{ENO} > \text{"E6"}}(\mathbf{EMP})$



Example: Final Result



...also, opportunities for parallel processing!

Vertical Fragmentation: Revisit

PERSON

<u>NAME</u>	ADDRESS	PHONE	SAL
John Smith	44, Here St	3456 7890	34000
Alex Brown	1, High St	3678 1234	48000
Harry Potter	99, Magic St	9976 4321	98000
Jane Morgan	87, Riverview	8765 1237	65800
Peter Jennings	65, Flag Rd	9851 1238	23980

PERSON1

<u>NAME</u>	ADDRESS	PHONE
John Smith	44, Here St	3456 7890
Alex Brown	1, High St	3678 1234
Harry Potter	99, Magic St	9976 4321
Jane Morgan	87, Riverview	8765 1237
Peter Jennings	65, Flag Rd	9851 1238

PERSON2

<u>NAME</u>	SAL
John Smith	34000
Alex Brown	48000
Harry Potter	98000
Jane Morgan	65800
Peter Jennings	23980

...primary Key [NAME] is included in all fragments

Reduction for VF (Vertical Fragmentation)

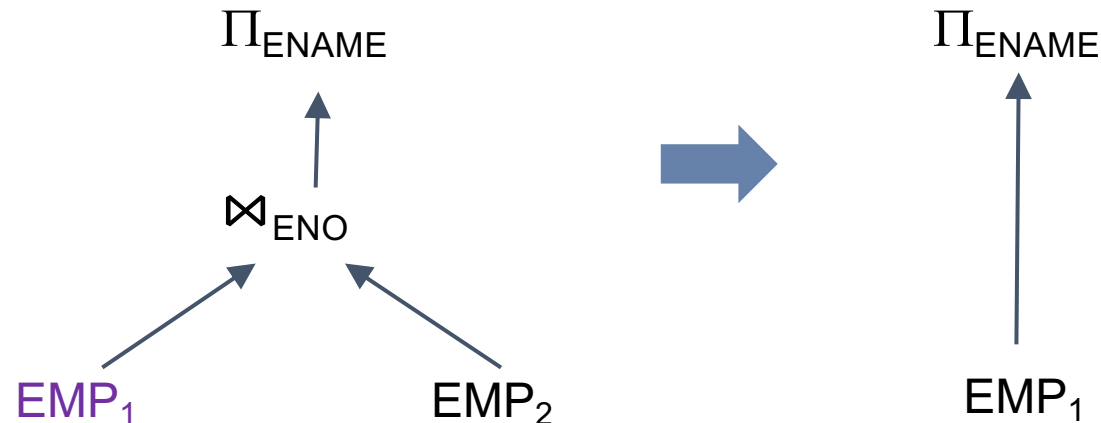
Find **useless intermediate relations**

Relation R defined over attributes $A = \{A_1, \dots, A_n\}$ vertically fragmented as $R_i = \Pi_{A'}(R)$ where $A' \subseteq A$:

Rule 3: $\Pi_D(R_i)$ is useless if the set of projection attributes D is not in A'

$EMP_1 = \Pi_{ENO, ENAME}(EMP); \quad EMP_2 = \Pi_{ENO, TITLE}(EMP)$

SELECT ENAME
FROM EMP

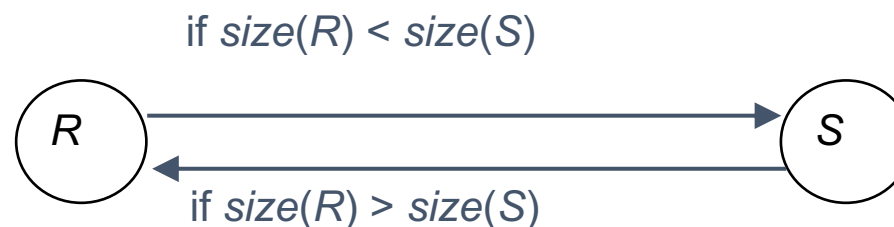


Join ordering and cost evaluation

Find the optimal way to join multiple distributed relations is very expensive and difficult because there are too many alternatives.

(transformation rules: associativity and commutativity)

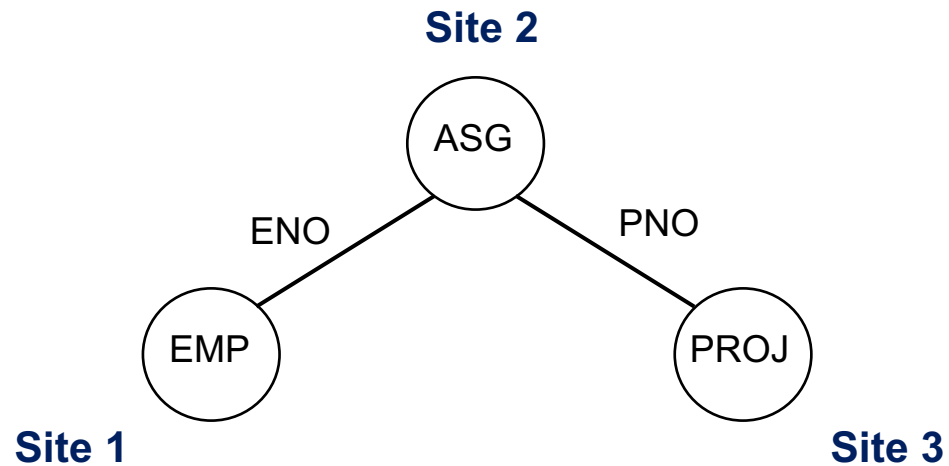
- **Model-based:** Compute the cost of all alternatives and select the best one
 - Necessary to compute the size of intermediate relations which is difficult.
- **Rule-based:** Use heuristics
 - Example: two distributed relations to be joined as a Search Space:



Join Ordering – Example

Consider how many ways the join can be executed?

$\text{PROJ} \bowtie_{\text{PNO}} \text{ASG} \bowtie_{\text{ENO}} \text{EMP}$

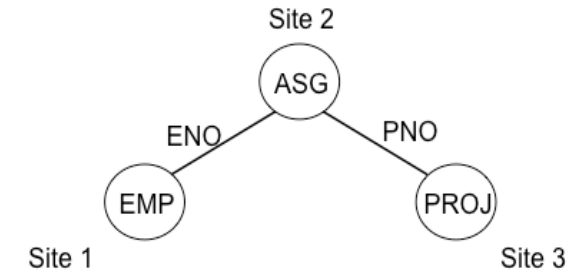


EMP			ASG			
<u>ENO</u>	ENAME	TITLE	<u>ENO</u>	<u>PNO</u>	RESP	DUR
E1	J. Doe	Elect. Eng.	E1	P1	Manager	12
E2	M. Smith	Syst. Anal.	E2	P1	Analyst	24
E3	A. Lee	Mech. Eng.	E2	P2	Analyst	6
E4	J. Miller	Programmer	E3	P3	Consultant	10
E5	B. Casey	Syst. Anal.	E3	P4	Engineer	48
E6	L. Chu	Elect. Eng.	E4	P2	Programmer	18
E7	R. Davis	Mech. Eng.	E5	P2	Manager	24
E8	J. Jones	Syst. Anal.	E6	P4	Manager	48
			E7	P3	Engineer	36
			E8	P3	Manager	40

PROJ			PAY	
<u>PNO</u>	PNAME	BUDGET	<u>TITLE</u>	SAL
P1	Instrumentation	150000	Elect. Eng.	40000
P2	Database Develop.	135000	Syst. Anal.	34000
P3	CAD/CAM	250000	Mech. Eng.	27000
P4	Maintenance	310000	Programmer	24000

Join Ordering – Example

Execution alternatives:

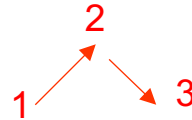


1. Site 1: **EMP** → **Site 2**

Site 2: computes $EMP' = EMP \bowtie ASG$

Site 2: **EMP'** → **Site 3**

Site 3 computes $EMP' \bowtie PROJ$

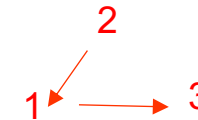


2. Site 2: **ASG** → **Site 1**

Site 1: computes $EMP' = EMP \bowtie ASG$

Site 1: **EMP'** → **Site 3**

Site 3: computes $EMP' \bowtie PROJ$

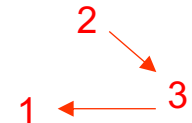


3. Site 2: **ASG** → **Site 3**

Site 3 computes $ASG' = ASG \bowtie PROJ$

Site 3: **ASG'** → **Site 1**

Site 1: computes $ASG' \bowtie EMP$

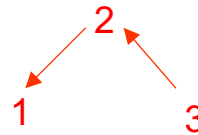


4. Site 3: **PROJ** → **Site 2**

Site 2: computes $PROJ' = PROJ \bowtie ASG$

Site 2: **PROJ'** → **Site 1**

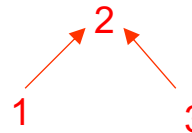
Site 1: computes $PROJ' \bowtie EMP$



5. Site 1: **EMP** → **Site 2**

Site 3: **PROJ** → **Site 2**

Site 2: computes $EMP \bowtie PROJ \bowtie ASG$



Required info: $size(EMP)$, $size(ASG)$, $size(PROJ)$, $size(EMP \bowtie ASG)$, and $size(ASG \bowtie PROJ)$

Are there any more alternatives?

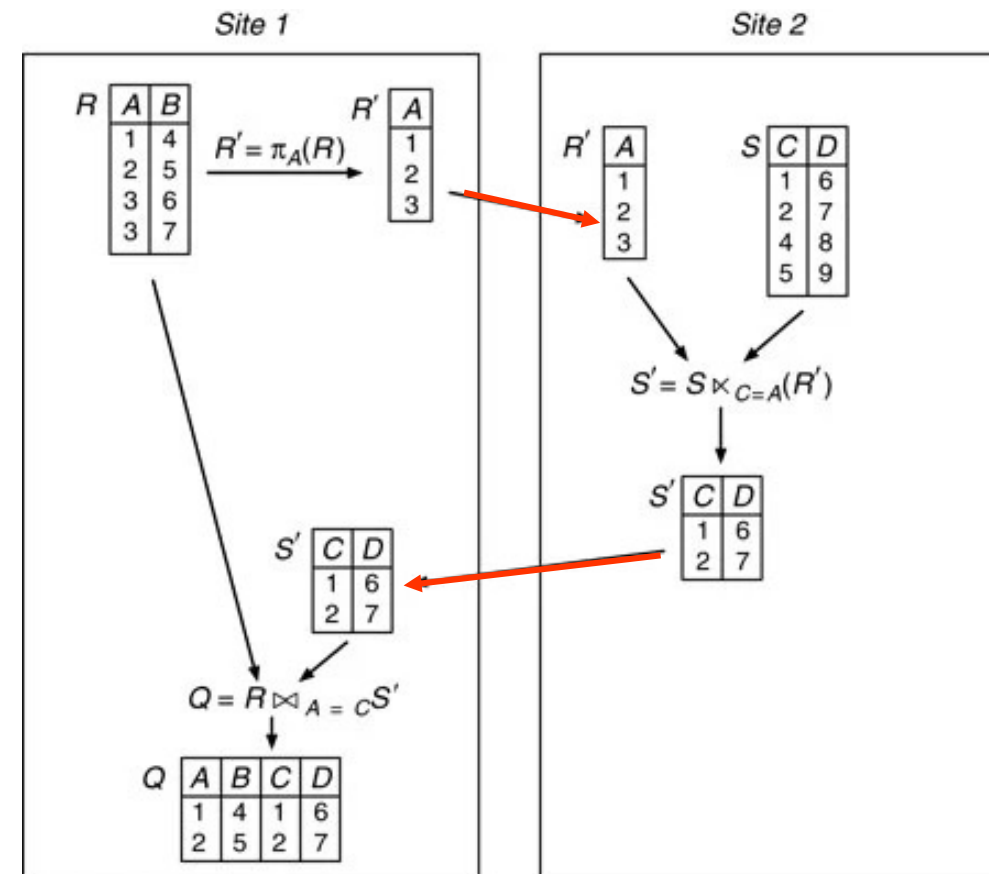
Local Query Optimization with Semi-Join

- Local query optimization can be performed **independently** with/without central control information
- Either a **single site or several sites** may participate in local query optimization
- Use **Semi-Join** to minimize the network data transmissions

$$R \bowtie_A S \Leftrightarrow (R \bowtie_A S) \bowtie_A S$$

$$\Leftrightarrow R \bowtie_A (S \bowtie_A R)$$

$$\Leftrightarrow (R \bowtie_A S) \bowtie_A (S \bowtie_A R)$$



Semi-join Algorithm

Note: Query Expression: **A:** ENO, **B:** PNO, **C:** PNAME="CAD/CAM" ^ ENO ≤ "E3", **E:** ENAME.

Query (Site 1): Find the names of Employees who are doing CAD/CAM projects and whose Employee number is less or equal than E3.

Option 1: Natural join

- Site 2: sends S and T to Site 1
- Site 1: computes $\Pi_E R \bowtie_A (\sigma_C(S \bowtie_B T))$

```
SELECT ENAME
FROM EMP, ASG, PROJ
WHERE EMP.ENO = ASG.ENO AND
      ASG.PNO = PROJ.PNO AND
      PNAME = "CAD/CAM" AND
      ASG.ENO <= "E3"
```

Option 2: **Semi-join** $\Pi_E R \bowtie_A (R \ltimes_A \sigma_C(S \bowtie_B T))$

- Site 1: $R' = \Pi_A(R)$
- Site 1: $R' \rightarrow$ Site 2
- **Site 2:** computes $S' = \sigma_C(S \bowtie_B T)$, $S'' = R' \ltimes_A S'$
- Site 2: $S'' \rightarrow$ Site 1
- **Site 1:** computes $\Pi_E R \bowtie_A S''$

Semi-join is better (**beneficial**) if:

$$\text{size}(\Pi_A(R)) + \text{size}(S \ltimes_A R') < \text{size}(S)$$

Site 1 (R)			Site 2 (S)			
EMP			ASG			
ENO	ENAME	TITLE	ENO	PNO	RESP	DUR
E1	J. Doe	Elect. Eng	E1	P1	Manager	12
E2	M. Smith	Syst. Anal.	E2	P1	Analyst	24
E3	A. Lee	Mech. Eng.	E2	P2	Analyst	6
E4	J. Miller	Programmer	E3	P3	Consultant	10
E5	B. Casey	Syst. Anal.	E3	P4	Engineer	48
E6	L. Chu	Elect. Eng.	E4	P2	Programmer	18
E7	R. Davis	Mech. Eng.	E5	P2	Manager	24
E8	J. Jones	Syst. Anal.	E6	P4	Manager	48
			E7	P3	Engineer	36
			E8	P3	Manager	40

Site 2 (T)			PAY	
PROJ			TITLE	SAL
PNO	PNAME	BUDGET		
P1	Instrumentation	150000	Elect. Eng.	40000
P2	Database Develop.	135000	Syst. Anal.	34000
P3	CAD/CAM	250000	Mech. Eng.	27000
P4	Maintenance	310000	Programmer	24000

Semi-Join ✕

Semi-join processes join between two tables that are stored on different sites. It is to reduce the transfer cost by first sending only the projected join column(s) to the other site, where it is joined with the second relation. Then, all matching tuples from the second relation are sent back to the first site to compute the final join result.

Q: Why is Semi-Join an important approach in query optimization?

A: Because Semi-Join can save the transfer cost during the join operations that require multiple fragments from multiple sites.

Q: Why can Semi-join save the transfer cost?

A: In an RDB model, tuples are organized according to functional dependencies, typically around primary keys. Therefore, when using a semi-join with foreign key relationships, it is not necessary to transfer *all* attribute values of every matching tuple. Instead, the system only needs to send the *key values* (i.e., the necessary identifiers) for those tuples that satisfy the join condition. This approach significantly reduces the data transferred across the network.

Summary

Distributed Query Processing

- Queries should be executed as local as possible, communications cost as little as possible
- Alternative QEPs (Query Execution Plans) are used to optimize the operations
 - Model-base vs Rule (Heuristics)-based optimization strategies
 - Do unary operations as early as possible
 - Do most restrictive operations as earlier as possible
 - Generate intermediate tuples as small as possible
 - Do expensive (join) operations as later as possible

Search Space is about all possible query execution plans and can be defined as the set of equivalent operator trees that can be produced using transformation rules.

- **GLOBAL QUERY OPTIMIZATION:** Construction of a Search Space should be as small as possible

Solution space is a subspace of Search Space

Total Cost vs Response Time: both should be considered

Key concepts: *Mutually Exclusive Fragments, Search Space, Solution Space, QEP, Relational Calculus, Relational Algebra, Cost Optimization, Semi-Join*

Reasoning is a key to learn this course